Releasing

Super Submitter for macOS. One export, two web pages, and a handful of commands.

Sending the code to GitHub builds a signed, notarized release and publishes it. The app then finds that release on its own, because Sparkle polls the appcast the release carries.

Nothing ships without you. Every run stops and waits for your approval. You press a button on the Actions page, and only then does it build.

Already set up. You do not need to do any of this.

- The repository **rafacst/app-super-submitter** exists and is public.
- Your Mac already knows how to reach it. The remote is named `github`.
- The approval gate exists. The environment is named `release` and you are the required reviewer.
- The Sparkle signing key already exists in your login keychain, and the app already carries its matching public half. **Never create a new one.**
- Your Mac is already signed in to GitHub, for Git and for the `gh` command. Nothing here asks for a GitHub password.

What is left is five secrets, one push, and one approval. Steps 1 to 3 happen once. Steps 4 and 5 are how you ship, every time.

The repository name is permanent. Its address is inside every copy of the app you ever release, and a released copy asks that address for updates forever. If you rename or delete the repository, every installed copy stops updating and there is no way to tell it where to look. This already happened once, to the previous repository. Leave the name alone.

Set it up once

1 Export the signing certificate Keychain Access

This is the Apple certificate that tells macOS the app is really yours. It is the one thing here with no command line worth writing: the Terminal can only export every private key you own at once, and you want exactly this one.

1. Press **Command** and **Space**, type **Keychain Access**, and press Return.
2. In the sidebar choose the **login** keychain, then the **My Certificates** category. This category matters. The plain **Certificates** category leaves out the private half, and the export will not work.
3. Find **Developer ID Application: R CASTRO SILVA CONSULTORIA EM TECNOLOGIA LTDA (88BXH8KNVZ)**.
4. Right-click it and choose **Export**.
5. Set the file format to **Personal Information Exchange (.p12)**. Save it to your Desktop as **certificate.p12**.
6. Your Mac asks you to invent a password for the file. Type one and write it down. You need it in step 3, as `DEVELOPER_ID_CERT_PASSWORD`.
7. Your Mac then asks for your login password, to allow the export. Type it and choose **Allow**.

2 Make an app-specific password appleid.apple.com

Apple checks every release before people can open it. That check signs in as you, and it needs a password of its own. Your real Apple ID password will not work.

1. Go to appleid.apple.com and sign in.
2. Open **Sign-In and Security**, then **App-Specific Passwords**.
3. Choose the **+** button, name it **Super Submitter releases**, and confirm.

4. Copy the password Apple shows you. It looks like `abcd-efgh-ijkl-mnop`. Apple shows it once only.

You need it in step 3, as `NOTARY_PASSWORD`.

3 Add the five secrets Terminal

The `gh` command puts the secrets on GitHub without a browser, and without the Sparkle key or the certificate ever reaching the clipboard. Open the Terminal app and go to the project folder. Every command below runs from there.

```
cd ~/apps/app-super-submitter-app
```

The Sparkle key. This key proves an update came from you, and the app refuses any download it did not sign. It is already in your login keychain, so read it from there.

```
SPARKLE_KEY=$(security find-generic-password \
  -s "https://sparkle-project.org" -a ed25519 -w)
echo "${#SPARKLE_KEY} characters"
```

If your Mac asks for your login password, type it and choose **Allow**. The answer must read **44 characters**. If it reads 0, the key was not read, and sending it now would set an empty secret. Run both lines again. Once it reads 44, send it:

```
printf '%s' "$SPARKLE_KEY" | gh secret set SPARKLE_PRIVATE_KEY
```

> **This key cannot be replaced.** If you ever lose it, no copy of the app that people already have can update again, ever. Back up that keychain item. Never run Sparkle's key generator a second time: a new key makes every installed copy refuse every update you publish, silently.

The certificate. A GitHub secret holds text, and `certificate.p12` is not text, so this turns it into text on the way. The second line deletes the file outright. GitHub holds it now, and a copy sitting on the Desktop is a copy that can be taken.

```
base64 -i ~/Desktop/certificate.p12 | gh secret set DEVELOPER_ID_CERT_P12
rm ~/Desktop/certificate.p12
```

The three you type. Run them one at a time. Each answers with *Paste your secret*. Paste the value and press Return. Nothing appears as you paste, which is normal. The first takes the file password you invented in step 1, the second your Apple ID, and the third the app-specific password from step 2.

```
gh secret set DEVELOPER_ID_CERT_PASSWORD   # from step 1
gh secret set NOTARY_APPLE_ID              # your Apple ID email
gh secret set NOTARY_PASSWORD              # from step 2
```

Now check your work. This prints the names only, never the values, and all five must be there.

```
gh secret list
```

Ship a release

4 Send the code to GitHub Terminal

```
cd ~/apps/app-super-submitter-app
git push github main
```

Name `github` every time. The other remote is `origin`, that one is GitLab, and it publishes nothing. No password is asked for. Your Mac is already signed in.

Xcode does the same job if you prefer it: the **Source Control** menu, then **Push…**, with the destination set to **github / main**.

5 Approve the release github.com

This one is a web page. The approval is the gate, and GitHub only opens it from the browser.

The tests run first, on their own, and they take a few minutes. GitHub asks for your approval only after they pass, so a red suite never reaches you as a decision.

1. Go to the **Actions** tab of the repository. A run named **Release** is waiting.
2. Open it. It says *Review pending deployments*.
3. Choose **Review deployments**, tick **release**, and choose **Approve and deploy**.

About fifteen minutes later the release appears on the Releases page, and every copy of the app already out there will offer it to its owner within a day.

To watch it without the browser, run this. Each step reports as it finishes, and the command ends when the run does. If a step turns red, it prints the failure: the workflow is written to say what went wrong in plain words.

```
gh run watch
```

Every release after the first

Repeat steps 4 and 5. Nothing else.

You can also start a release without sending any new code. It still waits for your approval.

```
gh workflow run release.yml
```

To change the version number people see, open **project.yml** and edit the `MARKETING_VERSION` line. The build number takes care of itself: it counts the commits, and that count is what tells the app a newer version exists.

What the release does, in order

1. Runs every test. Publishing cannot be undone, so nothing is signed until the tests pass.

2. Builds the app and signs it with your Developer ID certificate.
3. Sends it to Apple to be checked, and staples Apple's approval into the app.
4. Checks four things before it goes out: the build number reached the app, the licensing settings are filled in, the update address matches this repository, and the update list is signed.
5. Writes the release notes from your commit messages since the last release.
6. Signs the update list with your Sparkle key.
7. Publishes the app and the update list together as one GitHub release.

Things that break, and why

Every install stops updating, and nothing says so.

The repository was renamed or deleted. Its address is inside every copy already released and cannot be changed afterwards. There is no repair. Do not rename or delete it.

Every update is refused, and nothing says so.

Somebody made a new Sparkle key. The app only trusts the original. Put the original private key back in the login keychain. Nothing else fixes it.

Apple's check fails.

Usually the app-specific password from step 2 expired or was revoked. Make a new one, then run `gh secret set NOTARY_PASSWORD` again. Setting a secret twice replaces it. If the message instead names an entitlement, the app asked for a permission it has no profile for, and that is a code change.

The run fails on the signing certificate.

Developer ID certificates expire after five years. Renew it in your Apple Developer account, then redo step 1 and the certificate part of step 3 with the new one.

A `gh` command says you are not logged in.

The GitHub sign-in on your Mac expired. Run `gh auth login`, choose **`GitHub.com`**, then **`HTTPS`**, and follow the questions. It ends in a browser page

that you confirm.

A `gh` command names the wrong repository, or none.

You are not in the project folder. Run `cd ~/apps/app-super-submitter-app` first. To be certain, add `-R rafacst/app-super-submitter` to the end of any `gh` command.

The run never starts, or never pauses for you.

The `release` environment lost its reviewer. Open **Settings**, then **Environments**, then **release**, and put yourself back under **Required reviewers**.

The build fails and names a macOS version.

GitHub retired the machine the workflow asks for. Open `.github/workflows/release.yml`, find the `runs-on` line, and change it to the current macOS label.
